A MAINOR PROJECT REPORT ON

U-NET BASED DEEP LEARNING MODEL FOR RETINAL VESSEL SEGMENTATION

Submitted to partial fulfillment of the requirements for the award of the degree of

MASTER OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

By

JETTIGARI LOKESH GOUD (25911D5808)

Under the Guidance of

Dr. Ravi Mathey

Professor

Department of Computer Science and Engineering

VIDYA JYOTHI INSTITUTE OF TECHNOLOGY

(An Autonomous Institution)

(Approved by AICTE, Accredited by NAAC, NBA & permanently Affiliated to JNTUH, Hyderabad)

Aziz Nagar Gate, C.B. Post, Hyderabad-500075

2025-26

(Approved by AICTE, Accredited by NAAC, NBA & permanently Affiliated to JNTUHI, Hyderabad)
Aziz Nagar Gate, C.B. Post, Hyderabad-500075

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the project report titled " U-NET BASED DEEP LEARNING MODEL FOR RETINAL VESSEL SEGMENTATION" is being submitted by **JETTIGARI LOKESH GOUD (25911D5808)** in partial fulfillment for the award of the Degree of Master of Technology in Computer Science and Engineering, is a record of bonafide work carried out by him under my guidance and supervision. These results embodied in this project report have not been submitted to any other University or Institute for the award of any degree or diploma.

Internal Guide
Dr Ravi Mathey
Professor

Head of Department
Dr. D. Aruna Kumari
Professor

External Examiner

DECLARATION

I, **JETTIGARI LOKESH GOUD**, hereby declare that the project entitled **"U-NET BASED DEEP LEARNING MODEL FOR RETINAL VESSEL SEGMENTATION",** submitted in partial fulfilment of the requirements for the Master of Technology degree in Computer Science and Engineering, is my original work. This project has not been previously submitted for the award of any degree or qualification at any other institution.

Date: **JETTIGARI LOKESH GOUD (25911D5808)**

Place: Hyderabad

ACKNOWLEDGEMENT

I would like to express my heartfelt gratitude to my project guide, **Dr Ravi Mathey**, Professor at Vidya Jyothi Institute of Technology, Hyderabad, for his invaluable support, guidance, and insights throughout the course of this project. His encouragement and expertise have been instrumental in helping me deepen my understanding and enhance my skills. I am truly grateful for his mentorship.

I would also like to extend my sincere thanks to **Dr. D. Aruna Kumari**, Professor and Head of the Department of Computer Science and Engineering, for her unwavering support and motivation during my academic journey. Her dedication to academic excellence has inspired me, and her assistance has been vital to the successful completion of this work.

I am especially thankful to my Principal, **Dr. A. Srujana**, for providing the necessary infrastructure and resources that made this project possible. Her commitment to fostering a supportive learning environment has enabled me to explore and develop my ideas more effectively.

Lastly, I would like to express my deep appreciation to my parents for their constant encouragement, as well as to the dedicated faculty members who have guided me throughout the course. Their contributions have greatly shaped my progress and have played a significant role in bringing me to this stage in my academic journey.

JETTIGARI LOKESH GOUD - 25911D5808

ABSTRACT

U-NET BASED DEEP LEARNING MODEL FOR RETINAL VESSEL SEGMENTATION

Automatic retinal blood vessel segmentation in fundus images plays an important role in the screening, analysis, and evaluation of various ophthalmic diseases. However, accurate segmentation of retinal vessels remains challenging because blood vessels have different widths, complex branching structures, varying contrast, and can be difficult to distinguish from the surrounding retinal background. In this project, a deep learning-based retinal vessel segmentation system is developed using a U-Net architecture and the publicly available DRIVE retinal fundus dataset. The objective is to accurately distinguish retinal blood vessels from the background while providing an efficient and accessible system for visualization and analysis.

The proposed system implements a complete image segmentation pipeline consisting of green-channel extraction, CLAHE enhancement, normalization, field-of-view processing, and 64×64 patch generation. The U-Net model is trained for 30 epochs, with the best model checkpoint obtained at epoch 24. During inference, image patches are processed and reconstructed to generate the final vessel probability map. Different inference thresholds are evaluated, and a threshold of 0.50 is selected based on the highest Dice performance.

The evaluated results demonstrate that the developed model achieves a Dice score of 82.63%, IoU of 70.46%, Precision of 82.27%, Recall of 83.26%, and Accuracy of 96.99%. The trained model is further used to generate predictions for all 20 DRIVE test images. To make the system practical and interactive, a Django-based web application is developed for retinal image upload and automated segmentation. The application provides the original retinal image, binary vessel segmentation, vessel overlay visualization, vessel coverage statistics, pixel intensity statistics, model probability statistics, vessel confidence distribution, and downloadable segmentation masks.

The developed system demonstrates the integration of deep learning-based retinal vessel segmentation with a web-based interface, providing an accessible platform for automated retinal vessel analysis and visualization.

INDEX

LIST OF FIGURES

CHAPTER – 1

INTRODUCTION

1.1. Background

Retinal fundus images capture the blood vessel network inside the human eye, and the structure, appearance, and distribution of these vessels provide valuable information for retinal image analysis and computer-aided ophthalmic applications. Automatic retinal vessel segmentation is therefore an important task in medical image processing and deep learning.
Vessel segmentation is challenging because vessels vary widely in width, form complex branching patterns, and often show low contrast against the background — thin vessels in particular are easy to miss, while larger vessels differ considerably in appearance. These variations make traditional image-processing techniques inconsistent.

Deep learning addresses this by automatically learning meaningful features from training data. Among segmentation architectures, U-Net is widely used for biomedical image segmentation due to its encoder-decoder structure, which combines high-level context with fine spatial detail — making it well suited to capturing both large and thin retinal vessels.

This project develops a U-Net-based retinal vessel segmentation system using the DRIVE dataset. Preprocessing (green-channel extraction, CLAHE enhancement, normalization, field-of-view processing) and patch generation (64×64 patches) prepare the images for training and inference. The trained model is evaluated using Dice, IoU, Precision, Recall, and Accuracy, with threshold optimization used to convert probability maps into binary masks. A Django web application completes the system, letting users upload images and view the segmentation, overlay, vessel coverage, pixel and probability statistics, confidence distribution, and download the resulting mask.

1.2. Problem Statement

Automatic retinal vessel segmentation faces several challenges that affect the accuracy and usability of segmentation systems:

- **Variation in vessel size:** from large primary vessels to very thin branches.
- **Low contrast:** vessel intensity can closely match surrounding tissue.

- **Complex, interconnected structures:** must be preserved during segmentation.
- **Thin vessels:** easily missed despite carrying important structural information.
- **Preprocessing needs:** illumination variation in raw images must be corrected before training.
- **Patch-based processing:** reduces computational cost but requires careful reconstruction of the full image from patches.
- **Threshold selection:** the model outputs probabilities, not binary values, so an appropriate cutoff must be chosen.
- **Accessibility:** a trained model alone requires technical expertise; a user-friendly interface is needed to make results usable.

An integrated system is therefore needed that combines effective preprocessing, deep learning-based segmentation, threshold optimization, evaluation, and an accessible visualization interface.

1.3. Objectives

The **U-Net Based Deep Learning Model for Retinal Vessel Segmentation** is developed with the following objectives:

- Develop an automated retinal vessel segmentation pipeline.
- Prepare images using green-channel extraction, CLAHE, normalization, and FOV processing.
- Implement patch-based processing (64×64 patches) with reconstruction at inference.
- Train a U-Net model for binary vessel/background classification.
- Optimize the segmentation threshold based on performance.
- Evaluate the model using Dice, IoU, Precision, Recall, and Accuracy.
- Generate predictions on the DRIVE test set.
- Build a Django web application for interactive image upload and inference.
- Provide visual and statistical analysis — original image, mask, overlay, coverage, pixel and probability statistics, and confidence distribution.
- Allow users to download the generated segmentation mask.

1.4. Scope of the Project

The scope of the project covers the development of a complete retinal vessel segmentation pipeline, starting from retinal image preprocessing and model training and extending to web-based inference and visualization.

1.4.1. Image Processing and Segmentation: Fundus image preprocessing (green-channel extraction, CLAHE, normalization, FOV processing), 64×64 patch generation, U-Net-based binary segmentation, patch reconstruction, and probability-to-binary mask conversion.

1.4.2. Model Training and Evaluation: Uses the DRIVE dataset, trained for 30 epochs with the best checkpoint selected, threshold evaluation with 0.50 chosen for inference, and evaluation via Dice, IoU, Precision, Recall, and Accuracy on the 20 DRIVE test images. The model achieved Dice 82.63%, IoU 70.46%, Precision 82.27%, Recall 83.26%, and Accuracy 96.99% at the selected threshold.

1.4.3. Web Application and Visualization: A Django application providing image upload, automated U-Net inference, original/binary/overlay visualization, vessel coverage and background percentage, pixel counts, image intensity statistics, model probability statistics, confidence distribution, and downloadable masks.

The scope of the project is focused on automated retinal vessel segmentation and analysis using the DRIVE dataset and the developed U-Net model. It is intended as a research and educational system for image segmentation and visualization and is not intended to replace professional medical diagnosis or clinical decision-making.

CHAPTER – 2

LITERATURE REVIEW

2.1. Introduction

Retinal vessel segmentation is an important research area in medical image processing, as the retinal vascular network carries structural information useful for analyzing retinal and systemic diseases. The goal is to separate blood vessels from surrounding tissue and produce a binary representation of the vascular network. Approaches have evolved from traditional image processing and machine learning to modern deep learning, with public datasets like DRIVE providing a common basis for training and evaluation.

2.2. Traditional Retinal Vessel Segmentation Methods

Early methods relied on image-processing techniques exploiting vessel intensity, orientation, contrast, and tubular structure, including thresholding, mathematical morphology, edge detection, matched filtering, region growing, vessel tracking, line/ridge detection, and hand-crafted feature extraction. These methods are computationally efficient and simple to implement, but their performance suffers from illumination variation, low-contrast vessels, pathological regions, and complex vascular structure.

2.3. Machine Learning-Based Approaches

Machine learning allows models to learn vessel and background characteristics from labelled images rather than relying entirely on fixed rules. Typical features include pixel intensity, local contrast, texture, gradient information, vessel orientation, and neighborhood characteristics, which are fed to classifiers for pixel-level vessel/background decisions. While this improves on some traditional techniques, performance still depends heavily on the quality of hand-crafted features, which may not adequately capture the complex appearance of vessels across scales and images.

2.4. Deep Learning for Retinal Vessel Segmentation

Deep learning, particularly Convolutional Neural Networks (CNNs), automatically learns hierarchical feature representations, removing the need for manual feature design. Lower layers capture edges and local structure, while deeper layers learn broader context — both needed since the retinal vasculature spans multiple scales. This makes deep learning especially suited

to retinal vessel segmentation, where a good architecture must preserve fine spatial detail while understanding global context.

2.5. U-Net Architecture

U-Net is a CNN originally designed for biomedical image segmentation, built from an encoder path and a decoder path linked by skip connections. The encoder extracts higher-level features while reducing spatial resolution; the decoder restores resolution to produce a full-size segmentation map. Skip connections transfer encoder feature information directly to corresponding decoder levels, preserving spatial detail that would otherwise be lost during downsampling:

Input Image → Encoder → Bottleneck → Decoder → Segmentation Mask

Since thin vessels can occupy very few pixels, this ability to retain spatial detail makes U-Net well suited to this project's requirements.

2.6. DRIVE Dataset

The Digital Retinal Images for Vessel Extraction (DRIVE) dataset is a widely used, publicly available benchmark providing retinal fundus images with manually annotated vessel masks and field-of-view information, making it suitable for supervised segmentation research. This project uses DRIVE as the primary dataset for training, evaluation, and test prediction, providing a standardized basis for assessing the developed pipeline.

2.7. Image Preprocessing Techniques

Image preprocessing is an important stage in retinal vessel segmentation because the quality and representation of the input image can influence model performance.

2.7.1. Green-Channel Extraction

The green channel provides the best contrast between vessels and background among RGB channels, giving a simplified grayscale representation for vessel analysis; the system uses it as part of preprocessing.

2.7.2. CLAHE Enhancement

Contrast Limited Adaptive Histogram Equalization improves local contrast without the excessive amplification seen in global histogram equalization, helping distinguish vessels in low-contrast regions.

2.7.3. Normalization

Pixel values are normalized to [0, 1] before being fed to the network, ensuring consistent input during training and inference.

2.7.4. Patch-Based Processing

Rather than processing whole images, the project uses 64×64 patches to learn local vessel structure while reducing computational load; predictions from multiple patches are reconstructed into the final segmentation map during inference.

2.8. Threshold Optimization

The network outputs per-pixel vessel probabilities that must be converted to a binary mask via a threshold. A low threshold increases false positives, while a high threshold risks missing thin or low-confidence vessels. The project systematically evaluates thresholds from 0.05 to 0.50 and selects 0.50 based on the best Dice performance, avoiding arbitrary threshold selection.

2.9. Evaluation Metrics

Different evaluation metrics are used to assess the quality of retinal vessel segmentation.

2.9.1. Dice Score

The Dice score measures the overlap between the predicted segmentation and the ground-truth vessel mask. It is particularly useful for segmentation problems because it evaluates the agreement between the two regions.

2.9.2. Intersection over Union

Intersection over Union (IoU) measures the ratio between the intersection of the predicted and ground-truth regions and their union. It provides another measure of segmentation overlap.

2.9.3. Precision

Precision measures the proportion of pixels predicted as vessels that are actually vessel pixels. It provides an indication of the false-positive segmentation rate.

2.9.4. Recall

Recall measures the proportion of actual vessel pixels that are correctly identified by the model. It is particularly important for evaluating whether vessel structures are being missed.

2.9.5. Accuracy

Accuracy measures the proportion of correctly classified pixels among all evaluated pixels. Although useful as a general measure, accuracy should be considered together with other segmentation metrics because retinal images contain substantially more background pixels than vessel pixels.

2.10. Web-Based Retinal Vessel Segmentation Systems

Trained models are more practical when accessible through an intuitive interface rather than raw scripts. This project integrates the trained U-Net into a Django web application, letting users upload a fundus image and view results in-browser, including the original retinal image, binary vessel segmentation, vessel overlay, vessel coverage and background percentage, pixel intensity statistics, model probability statistics, mean vessel confidence and confidence distribution, and segmentation mask download. This connects the deep learning model to a practical interface for inspecting and interpreting results.

2.11. Research Gap

The literature shows a progression from manually designed image-processing techniques to machine learning and deep learning, with traditional methods struggling on complex vessel structures and deep learning requiring careful preprocessing, training, and evaluation. A practical implementation needs more than a segmentation model alone — it requires a reproducible preprocessing pipeline, model evaluation, threshold selection, test prediction, and an accessible way to view results. This project addresses that gap by combining DRIVE-based preprocessing, U-Net segmentation, threshold optimization, quantitative evaluation, test-set prediction, and Django-based visualization into a single pipeline.

CHAPTER – 3

FEASIBILITY STUDY

3.1. Introduction

The feasibility study determines whether the proposed U-Net Based Deep Learning Model for Retinal Vessel Segmentation can be successfully developed and operated using the available software, hardware, dataset, and development resources.

The project includes retinal image preprocessing, patch generation, U-Net training, evaluation, test prediction, and integration with a Django web application. The feasibility of the system is evaluated in terms of technical, operational, economic, schedule, and security considerations.

3.2. Technical Feasibility

Technical feasibility evaluates whether the required technologies, development tools, hardware, and dataset are available for implementing the proposed system.

The project uses established open-source technologies including Python, PyTorch, OpenCV, NumPy, Django, Matplotlib, Git, and GitHub.

3.2.1. Software & Hardware Requirements

The project can be developed and executed using commonly available software tools.

Software / Technology	**Purpose**
Python	Core programming language
PyTorch	U-Net model development and inference
OpenCV	Image processing and image manipulation
NumPy	Numerical and array operations
Django	Web application development
Matplotlib	Visualization and preprocessing analysis

The system can operate on a standard computer. PyTorch supports both CPU and CUDA-enabled GPU execution. The implemented inference system can operate on the **CPU**, so dedicated GPU hardware is not mandatory.

3.2.2. Dataset and Model Feasibility

The project uses the publicly available **DRIVE retinal fundus dataset**, which provides retinal images and corresponding vessel annotations for supervised segmentation. Therefore, collecting and manually annotating a new dataset is not required.

A **U-Net architecture** is used for binary vessel segmentation. Images are processed using **64×64 patches**, reducing computational requirements. During inference, patch predictions are reconstructed to produce the final segmentation mask.

The trained model is stored as a checkpoint and loaded by the Django application for inference, eliminating the need to retrain the model for each uploaded image.

3.2.3 Web Application Feasibility

The trained model is integrated into a Django web application. The application supports the complete workflow:

Image Upload → Image Processing → U-Net Inference → Segmentation → Visualization → Statistical Analysis

3.3. Operational Feasibility

The system is operationally feasible because users can interact with the trained model through a **Django web interface** without directly executing the machine-learning code.

3.4. Economic Feasibility

The project is economically feasible because it primarily uses **open-source software and a publicly available dataset**. No specialized medical imaging equipment is required.

The main resources required are:

- Existing computer hardware
- Python and open-source libraries
- DRIVE dataset
- Internet access

- Development time

GPU hardware is optional because inference can be performed on a CPU.

3.5. Schedule Feasibility

The project can be developed through the following major stages:

1. Dataset preparation
2. Image preprocessing and patch generation
3. U-Net development
4. Model training
5. Evaluation and threshold optimization
6. Test-set prediction
7. Django integration
8. Visualization and testing
9. Documentation

The major implementation stages have already been completed, including **training, threshold optimization, test prediction, Django integration, and automated testing**. Therefore, the project is schedule-feasible.

3.6. Security and Risk Considerations

The application validates uploaded files by checking whether the image exists and can be successfully decoded before inference. The trained model checkpoint must be protected from accidental modification or deletion. User-uploaded images should also be handled carefully to avoid invalid input.

The primary risk is **model prediction accuracy**. False-positive and false-negative vessel classifications may occur, particularly for images that differ from the training data. Therefore, the system should be considered a research and visualization tool rather than an independent clinical diagnostic system.

CHAPTER – 4

SYSTEM REQUIREMENTS

4.1. Introduction

System requirements define the software, hardware, functional, and non-functional specifications necessary to develop and deploy the **U-Net Based Deep Learning Model for Retinal Vessel Segmentation**. The system integrates a deep learning pipeline trained on fundus images with a Django web framework for real-time inference, mask visualization, and statistical evaluation.

4.2. Existing vs. Proposed System

- **Existing System:** Relies on traditional image-processing methods (edge detection, thresholding, morphological operations, matched filtering, vessel tracking) and hand-crafted ML classifiers. These struggle with low-contrast capillaries, illumination variations, and complex retinal backgrounds.
- **Proposed System:** Employs an end-to-end deep learning U-Net architecture trained on the DRIVE dataset. It utilizes green-channel extraction, CLAHE, patch-based inference (64×64), and automated metric computation integrated into a Django interface.

Pipeline Workflow:

DRIVE Dataset → Preprocessing (CLAHE/Norm) → Patch Extraction → U-Net Model → Reconstruction → Django Interface

4.3. Functional Requirements (FR)

Requirement ID	Module	Operational Description
FR-01 – FR-02	Input & Validation	Secure retinal image upload with automatic format and decode validation.
FR-03 – FR-04	Preprocessing	Green-channel extraction, CLAHE, normalization, FOV masking, and 64×64 patching.

FR-05 – FR-08	Inference Engine	Load best U-Net checkpoint, compute vessel probability maps, threshold at 0.50, and reconstruct output.
FR-09 – FR-10	Visualization	Display original image, predicted binary mask, overlay visualization, and vessel coverage percentage.
FR-11 – FR-13	Statistical Analysis	Compute pixel intensity stats, model confidence metrics, and confidence distributions (Low: 0.50--0.75, Mid: 0.75--0.90, High: ≥ 0.90).
FR-14 – FR-16	Export & Evaluation	Mask export/download, full DRIVE test set batch evaluation (Dice, IoU, Precision, Recall, Accuracy).

4.4. Non-Functional Requirements (NFR)

- **Performance:** Supports patch-based inference on standard CPUs; automatically leverages CUDA-enabled GPUs for accelerated processing.
- **Usability: Streamlined web workflow:** *Upload* → *Segment* → *View Results* → *Analyze Statistics* → *Download Mask*.
- **Reliability & Security:** Rejects corrupt or invalid file formats safely; ensures trained model weights remain write-protected.
- **Maintainability & Scalability:** Modular codebase separating data pipelines, model definition, training, inference, and web interfaces to allow easy addition of future architectures.
- **Reproducibility:** Controlled environment via requirements.txt, Git versioning, and fixed seed configurations.

4.5. System Specifications

Hardware Specifications

- **Processor:** Modern multi-core CPU (Intel Core i5/AMD Ryzen 5 or higher).
- **Memory (RAM):** 8 GB minimum (16 GB recommended).
- **Storage:** 10 GB free space for dataset, model checkpoints, and logs.
- **GPU (Optional):** NVIDIA GPU with CUDA support for accelerated training and real-time inference.

Software Specifications

- **Operating System:** Windows 10/11, Linux (Ubuntu 20.04+), or macOS.
- **Programming Environment:** Python 3.8+.
- **Deep Learning Framework:** PyTorch & Torchvision.
- **Image Processing & Math:** OpenCV, NumPy, Matplotlib.
- **Web Framework:** Django.

4.6. Dataset, Model, & System Parameters

Parameter	**Configuration / Specification**
Dataset	DRIVE (Digital Retinal Images for Vessel Extraction) with manual annotations
Model Architecture	U-Net (Encoder-Decoder with Skip Connections)
Input Patch Dimensions	64×64 pixels
Training Epochs	30 Epochs (Optimal checkpoint achieved at Epoch 24)
Binarization Threshold	0.50 (Optimized probability cutoff)
Intended Scope	Research, educational, and experimental visualization (non-clinical diagnostic)

4.7. System Constraints

- **Domain Dependency:** Model performance is optimized for the DRIVE dataset; cross-dataset generalization may require fine-tuning.
- **Image Quality:** Severe retinal pathologies, extreme blur, or low illumination directly impact vessel boundary precision.
- **Deployment Scope:** Configured for local development servers rather than high-throughput production clinical environments.

CHAPTER – 5

SYSTEM DESIGN

5.1. Introduction

System design defines the structural architecture, functional components, data pipeline, and user interactions for the U-Net Based Retinal Vessel Segmentation System. The system integrates two core subsystems:

1. **Machine Learning Pipeline:** Data ingestion, image preprocessing, patch-based model training, quantitative validation, and threshold optimization.
2. **Django Web Application:** Image validation, sliding-window inference, mask reconstruction, vessel overlay generation, and statistical metrics visualization.

$$\text{DRIVE Dataset} \rightarrow \text{Preprocessing} \rightarrow \text{Patch Extraction} \rightarrow \text{U-Net Model} \rightarrow \text{Thresholding } (0.50) \rightarrow \text{Reconstruction} \rightarrow \text{Web UI}$$

5.2. Data Processing & Ingestion Pipeline

- **Green-Channel Extraction:** Isolate the green spectral channel ($\approx$ 540--570 nm), which exhibits the highest vascular absorption and optimal contrast against retinal background tissue.
- **CLAHE Enhancement:** Apply Contrast Limited Adaptive Histogram Equalization to locally enhance capillary details without amplifying background sensor noise.
- **Patch Generation Strategy:** Extract 64×64 pixel sub-regions to optimize GPU memory footprint and capture local vascular topology.
    - ***Training:*** Random/uniform patch extraction across annotated masks.
    - ***Web Inference:*** Overlapping sliding window with a stride of 16 pixels.
    - ***Batch Evaluation:*** Non-overlapping stride of 64 pixels.
- **Data Loader:** Maps raw image patches into normalized floating-point tensors in range $[0,1]$ alongside corresponding binary masks.

5.3. U-Net Model & Training Architecture

The system uses a symmetrical U-Net architecture suited for biomedical pixel-level segmentation:

- **Contracting (Encoder) Path:** Repeated blocks of two 3×3 convolutions, ReLU activation functions, and 2×2 max-pooling operations for hierarchical context extraction.
- **Bottleneck Layer:** Intermediate representation capturing deep semantic features at minimal spatial resolution.
- **Expanding (Decoder) Path:** 2×2 transposed convolutions that restore feature map resolution step-by-step.
- **Skip Connections:** Direct identity channels routing spatial details from encoder layers to decoder layers, preserving micro-vessel edge definition.
- **Output Layer:** 1×1 convolution activated via a Sigmoid function returning pixel probabilities $P(x, y) \in [0,1]$.

Training Parameters: Total Epochs: 30 | Best Checkpoint: Epoch 24 (best_unet.pth) | Loss: Binary Cross-Entropy / Dice Loss | Input Size: $64 \times 64 \times 1$.

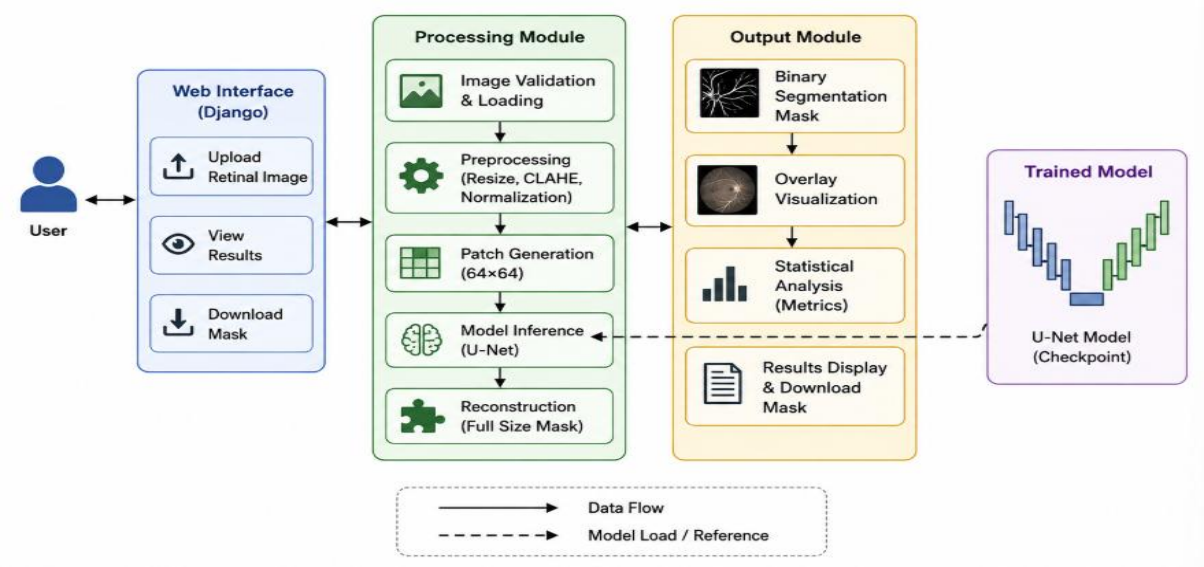

Fig 5.1: System Architecture

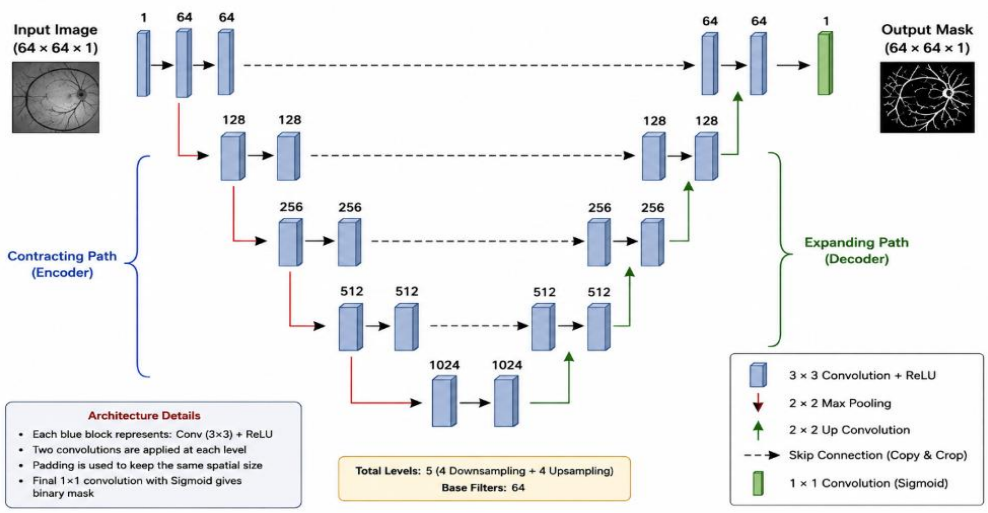

Fig 5.2: U-NET Model Architecture

5.4. Threshold Optimization & Evaluation

Predicted pixel probabilities are mapped to discrete binary categories using an empirical threshold $\tau = 0.50$:

$$\text{Class}(x, y) = \begin{cases} 1 \text{ (Vessel)}, & P(x, y) \geq 0.50 \\ 0 \text{ (Background)}, & P(x, y) < 0.50 \end{cases}$$

Metric	**Formula**	**Result**
Dice Score (F1)	$\frac{2\lvert P \cap G \rvert}{\lvert P \rvert + \lvert G \rvert}$	82.63%
Intersection over Union (IoU)	$\frac{\lvert P \cap G \rvert}{\lvert P \cup G \rvert}$	70.46%
Precision	$\frac{TP}{TP + FP}$	82.27%
Recall (Sensitivity)	$\frac{TP}{TP + FN}$	83.26%
Overall Accuracy	$\frac{TP + TN}{TP + TN + FP + FN}$	96.99%

5.5. Inference Pipeline & Web Workflow

Sliding-Window Extraction: Input images are padded and converted into overlapping 64×64 patches with a stride of 16 pixels.

Probability Accumulation: Forward-pass predictions are spatially accumulated and divided by an overlap-frequency weight map to eliminate boundary stitching artifacts.

Overlay Synthesis: The binary vessel mask is highlighted in distinct red pixels ($R = 255, G = 0, B = 0$) and blended directly onto the original fundus image.

5.6. Statistical Analytics & User Interface

The Django application generates interactive visualizations and automated tabular analytics:

- **Morphological Vessel Metrics:** Total vessel pixel count, background pixel count, and vessel coverage ratio:

$$A_v = \left(\frac{\text{Vessel Pixels}}{\text{Total Pixels}}\right) \times 100$$

- **Intensity Metrics:** Mean intensity, standard deviation, minimum, and maximum pixel values.
- **Confidence Distribution:** Vessel predictions are segmented into confidence intervals:
  - ***Low Confidence:*** $0.50 \leq P < 0.75$ (Faint peripheral micro-capillaries)
  - ***Medium Confidence****:* $0.75 \leq P < 0.90$ (Intermediate vascular branches)
  - ***High Confidence:*** $P \geq 0.90$ (Primary vascular trunks)
- **Export Utilities:** One-click lossless download option for reconstructed binary masks.

5.7. Database & Deployment Strategy

- **Stateless Inference:** Operates using an in-memory execution pipeline per HTTP request, eliminating database overhead for single-session inference.
- **Execution Environment:** Runs locally via python manage.py runserver at (http://127.0.0.1:8000/). Inference defaults to standard CPU execution with automatic CUDA GPU acceleration when supported.

CHAPTER – 6

SOFTWARE IMPLEMENTATION

6.1. Technologies Used

The system employs a full-stack deep learning and web development stack configured to execute end-to-end retinal vessel segmentation, model evaluation, and dynamic metric reporting.

Technology	Role & Primary Functionality
Python	Primary development language for deep learning, computer vision, and backend services.
PyTorch	Deep learning framework; manages tensor transformations, autograd, and U-Net training/inference.
OpenCV	Image reading, color-space conversion, CLAHE contrast enhancement, and overlay generation.
NumPy	Multi-dimensional array operations, probability matrix accumulation, and statistical computations.
Django	Web server framework; handles routing, multipart uploads, UI views, and template rendering.
Matplotlib	Data visualization for preprocessing benchmarks and evaluation metrics.
Git / GitHub	Distributed version control, codebase tracking, and deployment management.
HTML5 / CSS3	Frontend presentation layer for interactive uploads and diagnostic analytics dashboards.

6.2. Modular Architecture & Subsystem Description

- **Dataset Module:** Converts raw DRIVE images and binary annotations into normalized ($[0,1]$), PyTorch-compatible tensors in Channel-First format ($C \times H \times W$).
- **Preprocessing Module:** Extracts the green channel for optimal vascular optical density and applies CLAHE (clipLimit $= 2.0$,tileGridSize $= 8 \times 8$).

- **U-Net Model:** Symmetrical encoder-decoder architecture with skip connections. Logits pass through a Sigmoid activation function to yield continuous probability maps:

$$\hat{Y} = \sigma\big(\text{U-Net}(X)\big) \in [0,1]$$

- **Model Training & Evaluation:** Trains for 30 epochs, storing the optimal weights at Epoch 24 (best_unet.pth). It validates performance using standard segmentation metrics:

Metric	Dice (F1)	IoU	Precision	Recall	Accuracy	Optimal Threshold (τ)
Score	82.63%	70.46%	82.27%	83.26%	96.99%	0.50

- **nference Engine**: Loads best_unet.pth on available hardware (CUDA/CPU). It executes sliding-window patch extraction (64×64, stride $= 16$), accumulates overlapping window outputs, and averages them across a spatial weight matrix to remove edge artifacts.
- **Django Web Layer:** Direct in-memory request-response architecture. It intercepts uploads via request.FILES, decodes them with OpenCV, invokes VesselSegmenter, generates vessel metrics, and encodes outputs into Base64 for web rendering.

6.3. Frontend-Backend Integration

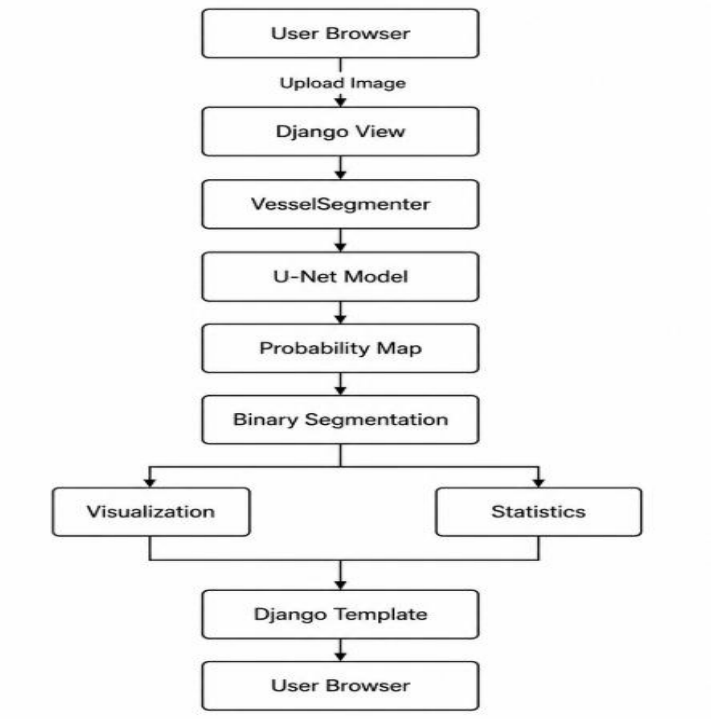

Fig 6.1: Frontend-Backend Integration

6.4. Sample Code Snippets

6.4.1. U-Net Model Initialization and Inference

Loading the Trained Model:

```
self.device = torch.device(
    "cuda"
    if torch.cuda.is_available()
    else "cpu"
)
self.model = UNet().to(
    self.device
)
checkpoint = torch.load(
    CHECKPOINT,
    map_location=self.device
)
self.model.load_state_dict(
    checkpoint["model_state_dict"]
)
self.model.eval()
```

6.4.2. Django Backend Implementation

```
uploaded_file = request.FILES.get(
    "retinal_image"
)
if not uploaded_file:
    return render(
        request,
```

```
            "segmentation/home.html",
            {
                "error":
                "Please select a retinal image."
            }
        )
    data = np.frombuffer(
        uploaded_file.read(),
        np.uint8
    )
    image = cv2.imdecode(
        data,
        cv2.IMREAD_COLOR
    )
    prediction, probability_map = (
        segmenter.predict(
            image,
            return_probability=True
        )
    )
```

6.4.3 Vessel Overlay Generation

```
    overlay = image.copy()

    overlay[prediction > 0] = [
        0,
        0,
        255
```

```
]
blended = cv2.addWeighted(
    image,
    0.7,
    overlay,
    0.3,
    0
)
```

6.4.4. Conversion of Results for Web Display

```
def image_to_base64(image):
    ok, encoded = cv2.imencode(
        ".png",
        image
    )
    if not ok:
        return None
    return base64.b64encode(
        encoded.tobytes()
    ).decode("utf-8")
```

CHAPTER – 7

SYSTEM TESTING

7.1. Introduction

System testing is performed to verify that the retinal vessel segmentation system works correctly from image input to final segmentation output. The testing process covers the preprocessing pipeline, U-Net model, prediction module, Django web application, and automated application tests.

The main objective is to ensure that the system produces valid segmentation results and that the web application handles retinal image uploads correctly.

7.2. Testing Objectives

The major objectives of testing are:

- To verify that retinal images are processed correctly.
- To verify that the U-Net model loads successfully.
- To verify that segmentation predictions are generated correctly.
- To verify the selected segmentation threshold.
- To verify test-set prediction generation.
- To verify the Django web application.
- To verify image upload and result display.
- To verify that invalid input is handled properly.
- To verify the basic functionality through automated tests.

7.3. Unit Testing

Individual components of the system are tested separately.

Test Component	**Testing Objective**	**Result**
Dataset Loader	Verify image and mask loading	Passed
Preprocessing	Verify image preprocessing operations	Passed
U-Net Model	Verify model initialization and loading	Passed
Inference	Verify segmentation prediction	Passed
Django Home View	Verify page loading	Passed

Image Upload	Verify uploaded image processing	Passed

7.4. Model Testing

The trained U-Net model was tested using retinal images after the training process.

The model uses a 64×64 patch size and reconstructs the individual patch predictions into a complete segmentation map.

Threshold testing was performed using values from 0.05 to 0.50. The selected threshold was 0.50.

The obtained evaluation results were:

Metric	**Result**
Dice Score	82.63%
IoU	70.46%
Precision	82.27%
Recall	83.26%
Accuracy	96.99%

These results indicate that the trained model is able to distinguish retinal vessels from the background with satisfactory segmentation performance.

7.5. Automated Testing

Automated tests were implemented using Django's TestCase framework.

Two tests were created:

- **Test 1 – Home Page Loading**

```
def test_home_page_loads(self):
    response = self.client.get(reverse("home"))
    self.assertEqual(response.status_code, 200)
```

This test verifies that the application's home page returns an HTTP 200 response.

- **Test 2 – Page Content**

```
def test_home_page_contains_title(self):
    response = self.client.get(reverse("home"))
    self.assertContains(response, "Retinal")
```

This verifies that the expected retinal segmentation content is present on the page.

7.6. Functional Test Cases

Test ID	Test Case	Expected Result	Status
TC01	Open application	Home page displayed	Passed
TC02	Upload valid retinal image	Image accepted	Passed
TC03	Generate segmentation	Vessel mask generated	Passed
TC04	Display original image	Original image displayed	Passed
TC05	Display vessel mask	Binary segmentation displayed	Passed
TC06	Display overlay	Vessel overlay displayed	Passed
TC07	Calculate vessel coverage	Coverage statistics displayed	Passed
TC08	Display probability statistics	Probability information displayed	Passed
TC09	Submit without image	Error message displayed	Passed
TC10	Django automated tests	All tests pass	Passed

7.7. System Testing Summary

The complete system was tested from model execution through web-based inference. The trained U-Net successfully generated retinal vessel segmentation predictions, and all 20 DRIVE test images were processed successfully.

The Django application was also verified using automated tests, with 2 out of 2 tests passing.

Therefore, the implemented system satisfies the functional requirements defined for retinal vessel segmentation and interactive web-based inference.

CHAPTER – 8

RESULT AND OUTPUT SCREENS

8.1. Introduction

The developed retinal vessel segmentation system was evaluated using the DRIVE retinal fundus dataset and the implemented Django web application. The results demonstrate the performance of the trained U-Net model and the functionality of the web-based segmentation system.

The system successfully performs retinal image preprocessing, U-Net-based vessel segmentation, prediction reconstruction, visualization, and statistical analysis.

8.2. Model Performance Results

The inference threshold was evaluated using different threshold values between 0.05 and 0.50. The best performance was obtained at a threshold of 0.50.

The final evaluation results are:

Metric	**Result**
Dice Score	82.63%
IoU	70.46%
Precision	82.27%
Recall	83.26%
Accuracy	96.99%
Selected Threshold	0.50

The results show that the trained U-Net model can effectively identify retinal blood vessels from the retinal background.

8.3. Test Prediction Results

- The trained model was applied to all 20 DRIVE test images. Each image was processed using patch-based inference and the resulting predictions were reconstructed into complete segmentation masks.

- The predicted vessel coverage across the test images ranged approximately from: 7.29% to 10.87%
- All 20 test images were successfully processed and corresponding prediction masks were generated.

8.4. Segmentation Output

The system generates a binary segmentation mask in which:

- White pixels represent predicted retinal vessels.
- Black pixels represent the background.

The prediction output can be used to visually examine the vessel structures detected by the U-Net model.

Figure 8.1: Retinal Vessel Segmentation Output

8.5. Vessel Overlay Output

The system also generates an overlay visualization by highlighting the predicted vessels on the original retinal image.

The overlay provides an intuitive representation of the segmentation result and makes it easier to compare the detected vessels with the original fundus image.

Figure 8.2 shows the predicted retinal vessels highlighted over the original retinal fundus image.

8.6. Django Web Application Output

The Django web application provides an interactive interface for uploading retinal images and viewing the segmentation results.

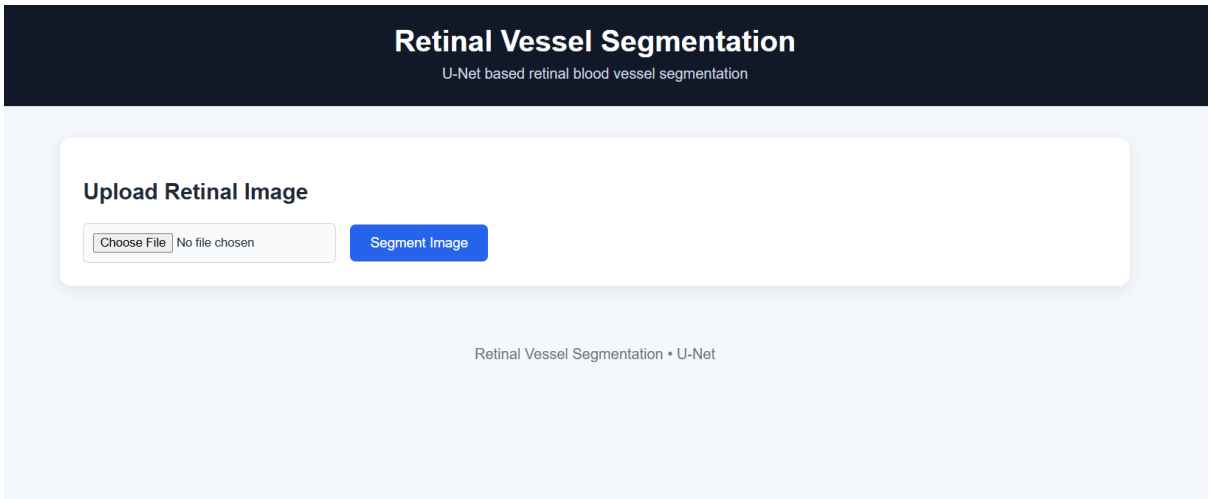

Figure 8.3: Django Retinal Vessel Segmentation Interface

8.7. Image Statistics Output

These values provide additional information about the input image and the generated segmentation.

Figure 8.4: Image and Segmentation Statistics

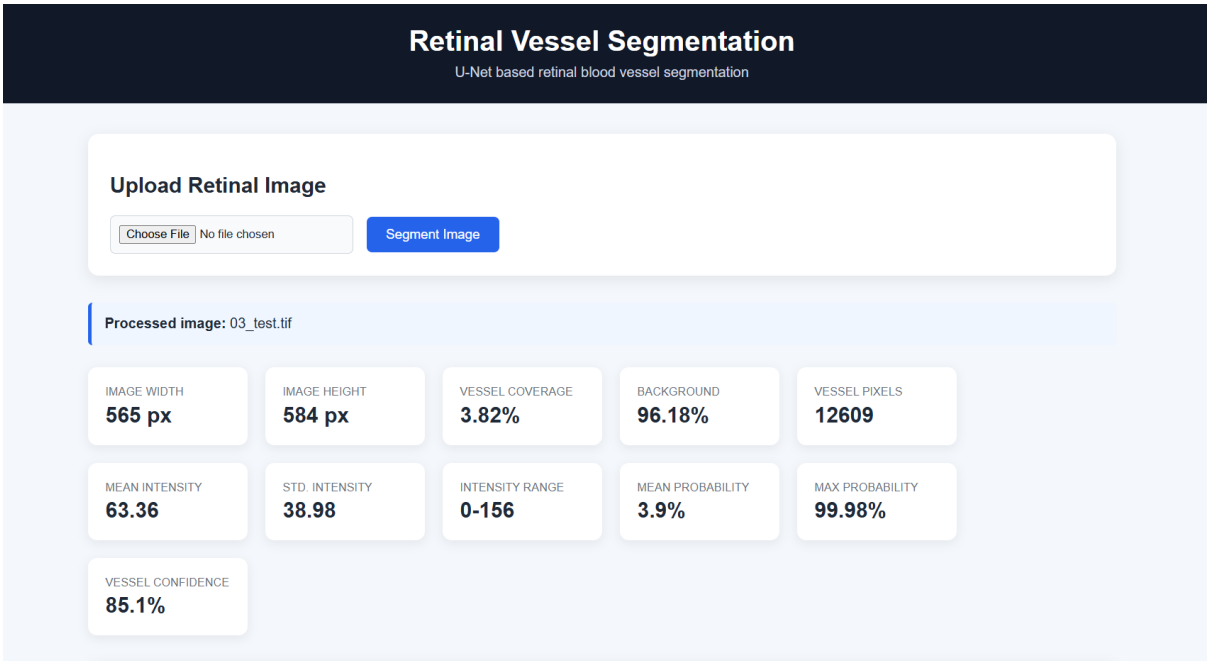

Figure 8.4: Image and Segmentation Statistics

8.8. Model Probability and Confidence Output

The application also provides statistics derived from the U-Net probability map.

These values provide an indication of the model's confidence in the predicted vessel regions.

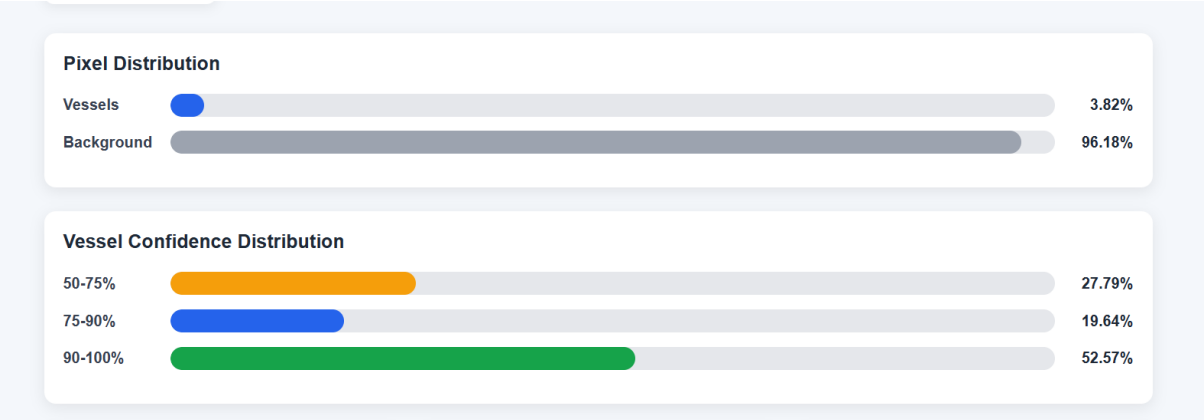

Figure 8.5: Model Probability and Vessel Confidence Statistics

8.9. Segmentation Mask Download

The web application provides an option to download the generated segmentation mask. This allows the prediction to be saved and used for further analysis or comparison with other segmentation results.

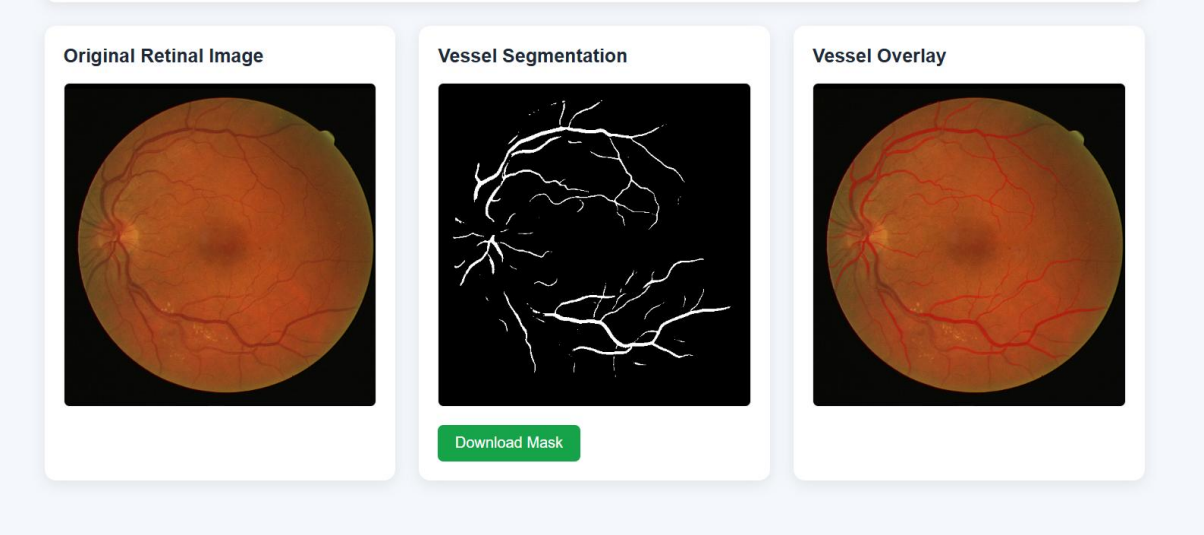

Figure 8.6: Segmentation Mask Download Optio

8.10. Overall Results

The developed system successfully integrates deep learning-based retinal vessel segmentation with a Django web interface.

The major results obtained are:

- A trained U-Net model was successfully developed.
- The best model checkpoint was obtained at epoch 24.
- The optimized segmentation threshold was 0.50.
- A Dice score of 82.63% was achieved.
- An IoU of 70.46% was achieved.
- All 20 DRIVE test images were successfully processed.
- The Django application successfully performs image upload and segmentation.
- Segmentation masks and vessel overlays are generated successfully.
- Image, probability, and confidence statistics are displayed.

CHAPTER – 9

CONCLUSION

The Multi-Level Attention Network for Segmenting Retinal Vessels project was developed to provide an automated and efficient approach for identifying retinal blood vessels from fundus images. Retinal vessel segmentation is an important task in ophthalmic image analysis because the structure, density, and abnormalities of retinal vessels can provide useful information for the study and assessment of various eye-related conditions. Manual identification of vessels is time-consuming and depends heavily on expert interpretation. Therefore, an automated deep learning-based approach can provide a useful solution for assisting retinal image analysis.

In this project, a complete retinal vessel segmentation pipeline was implemented using the DRIVE retinal fundus dataset. The pipeline includes image preprocessing, patch generation, dataset preparation, U-Net model training, model evaluation, threshold optimization, test-set prediction, and web-based inference. The preprocessing stage prepares retinal images for segmentation, while the patch-based approach allows the model to learn vessel patterns from smaller image regions.

A U-Net architecture was trained for binary retinal vessel segmentation. The model was trained for 30 epochs, with the best checkpoint obtained at epoch 24. The inference threshold was optimized by evaluating multiple threshold values, and 0.50 was selected based on the best Dice performance. At this threshold, the system achieved a Dice score of 82.63%, IoU of 70.46%, Precision of 82.27%, Recall of 83.26%, and Accuracy of 96.99%. These results demonstrate that the implemented model is capable of distinguishing retinal vessels from the surrounding background with satisfactory segmentation performance.

The trained model was further used to generate predictions for all 20 DRIVE test images. The system successfully processed the complete test set and generated corresponding binary vessel segmentation masks. The predicted vessel coverage ranged approximately from 7.29% to 10.87%, demonstrating that the system can produce quantitative information in addition to visual segmentation results.

To make the developed model accessible and practical, a Django-based web application was integrated with the trained U-Net model. The application allows users to upload retinal fundus images and obtain segmentation results directly through a web interface. It displays the original image, binary segmentation mask, and vessel overlay. In addition, the application calculates

vessel coverage, background percentage, pixel intensity statistics, model probability statistics, and vessel confidence distribution. The generated segmentation mask can also be downloaded for further analysis.

Automated testing was incorporated into the Django application to verify important application functionality. The implemented tests were executed successfully, with 2 out of 2 tests passing, providing additional confidence in the basic operation of the web interface.

Although the current system provides promising results, there are opportunities for further improvement. The model can be enhanced by using more advanced architectures, attention mechanisms, multi-scale feature extraction, improved data augmentation, and larger retinal image datasets. Performance could also be evaluated using additional publicly available datasets such as STARE and CHASE_DB1 to study the generalization capability of the model. Further optimization could improve the segmentation of very thin vessels, low-contrast vessels, and complex retinal structures.

In conclusion, the project successfully demonstrates the integration of deep learning, image processing, model evaluation, and web application development into a functional retinal vessel segmentation system. The developed system provides both qualitative visualization and quantitative analysis of retinal vessels and establishes a strong foundation for future research and development in automated retinal image analysis.

CHAPTER – 10

FUTURE ENHANCEMENTS

Although the Multi-Level Attention Network for Segmenting Retinal Vessels successfully provides automated retinal vessel segmentation, several improvements can be introduced in future versions to increase segmentation accuracy, generalization, usability, and practical applicability.

10.1. Advanced Deep Learning Architectures

The current system uses a U-Net-based architecture for retinal vessel segmentation. Future work can investigate more advanced segmentation architectures incorporating attention mechanisms, residual connections, dense feature extraction, and multi-scale feature learning. Architectures such as Attention U-Net, U-Net++, and other encoder-decoder networks could be evaluated to improve the detection of thin and low-contrast retinal vessels.

10.2. Multi-Level and Multi-Scale Feature Extraction

Retinal vessels have significantly different widths and structures. Future versions can introduce stronger multi-scale feature extraction to capture both large vessels and very thin capillaries. Combining features from different levels of the network can improve boundary detection and preserve fine vessel structures.

10.3. Improved Preprocessing

The current preprocessing pipeline uses green-channel extraction, CLAHE enhancement, normalization, and FOV masking. Future work can investigate additional preprocessing techniques such as advanced illumination correction, noise reduction, color normalization, and adaptive enhancement methods. These techniques may improve vessel visibility before segmentation.

10.4. Data Augmentation

The DRIVE dataset contains a relatively limited number of retinal images. Future development can use more extensive data augmentation techniques such as rotation, flipping, scaling, cropping, brightness adjustment, contrast modification, and elastic transformations. This can increase training diversity and reduce overfitting.

10.5. Training with Additional Datasets

Future versions can be trained and evaluated using additional publicly available retinal datasets such as STARE and CHASE_DB1. Using multiple datasets would provide greater variation in retinal appearance and vessel structures and would allow the model's generalization capability to be evaluated more thoroughly.

10.6. Improved Segmentation of Thin Vessels

One of the major challenges in retinal vessel segmentation is accurately detecting extremely thin vessels. Future work can introduce specialized loss functions, boundary-aware learning, attention mechanisms, or class-balancing techniques to improve the detection of small vessel structures while reducing false positives.

10.7. Model Optimization and Faster Inference

The current application performs inference using the trained model on the available computing device. Future development can optimize the model for faster inference through techniques such as model pruning, quantization, knowledge distillation, and optimized deployment formats. This would make the system more suitable for real-time or resource-constrained environments.

10.8. Enhanced Web Application

The Django application can be expanded with additional features such as user authentication, image history, comparison of multiple predictions, result management, and interactive visualization. Users could also be provided with side-by-side comparisons of the original retinal image, ground-truth mask, predicted mask, and vessel overlay when reference annotations are available.

10.9. Overall Future Scope

The developed project provides a foundation for further research in automated retinal image analysis. Future enhancements can focus on improving segmentation accuracy, particularly for thin vessels, increasing model generalization across different datasets, reducing inference time, and extending the web application with advanced analytical capabilities. With these improvements, the system can evolve from a retinal vessel segmentation prototype into a more comprehensive platform for automated retinal image analysis and research-oriented clinical decision support.

CHAPTER – 11

REFERENCES

[1] Staal, J., Abramoff, M. D., Niemeijer, M., Viergever, M. A., and van Ginneken, B. (2004). **Ridge-Based Vessel Segmentation in Color Images of the Retina.** *IEEE Transactions on Medical Imaging*, 23(4), 501–509. https://ieeexplore.ieee.org/document/1282003

[2] Ronneberger, O., Fischer, P., and Brox, T. (2015). **U-Net: Convolutional Networks for Biomedical Image Segmentation.** *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, 234–241. https://arxiv.org/abs/1505.04597

[3] Hoover, A., Kouznetsova, V., and Goldbaum, M. (2000). **Locating Blood Vessels in Retinal Images by Piecewise Threshold Probing of a Matched Filter Response.** *IEEE Transactions on Medical Imaging*, 19(3), 203–210. https://ieeexplore.ieee.org/document/841970

[4] Soares, J. V. B., Leandro, J. J. G., Cesar, R. M., Jelinek, H. F., and Cree, M. J. (2006). **Retinal Vessel Segmentation Using the 2-D Gabor Wavelet and Supervised Classification.** *IEEE Transactions on Medical Imaging*, 25(9), 1214–1222. https://ieeexplore.ieee.org/document/1677512

[5] Fraz, M. M., Remagnino, P., Hoppe, A., Uyyanonvara, B., Barman, S. A., and others. (2012). **An Ensemble Classification-Based Approach Applied to Retinal Vessel Segmentation.** *IEEE Transactions on Biomedical Engineering*, 59(9), 2538–2548. https://ieeexplore.ieee.org/document/6191478

[6] Liskowski, P., and Krawiec, K. (2016). **Segmenting Retinal Blood Vessels with Deep Neural Networks.** *IEEE Transactions on Medical Imaging*, 35(11), 2369–2380. https://ieeexplore.ieee.org/document/7481330

[7] **DRIVE – Digital Retinal Images for Vessel Extraction.** University Medical Center Utrecht. https://drive.grand-challenge.org/

[8] Tajbakhsh, N., Jeyaseelan, L., Li, Q., Chiang, J. N., Wu, Z., and Ding, X. (2016). **Embracing Imperfect Datasets: A Review of Deep Learning Solutions for Medical Image Segmentation.** *Medical Image Analysis*, 34, 76–93. https://doi.org/10.1016/j.media.2016.06.004

[9] Litjens, G., Kooi, T., Bejnordi, B. E., Setio, A. A. A., Ciompi, F., Ghafoorian, M., van der Laak, J. A. W. M., van Ginneken, B., and Sánchez, C. I. (2017). **A Survey on Deep Learning in Medical Image Analysis.** *Medical Image Analysis*, 42, 60–88. https://doi.org/10.1016/j.media.2017.07.005

[10] Siddique, N., Sidike, P., Elkin, C., and Devabhaktuni, V. (2021). **U-Net and Its Variants for Medical Image Segmentation: Theory and Applications.** https://arxiv.org/abs/2011.01118

[11] Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., et al. (2019). **PyTorch: An Imperative Style, High-Performance Deep Learning Library.** *Advances in Neural Information Processing Systems (NeurIPS)*, 32. https://papers.neurips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library